

Week 4

C for Embedded Systems

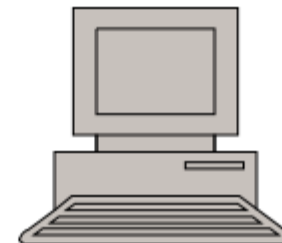
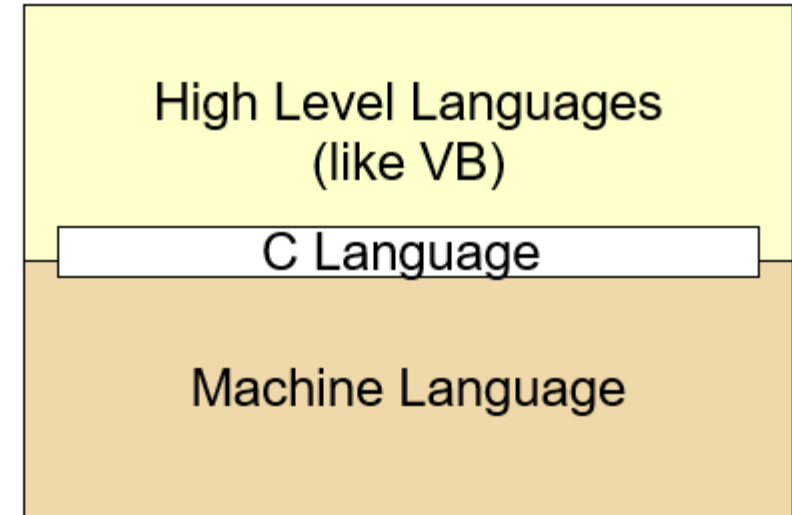
Winter 2026

Topics

- Assembler vs. Compiler
- Software Development
- C Program Structure
- C Data Types
- ANSI C ISO89 vs. ISO99 standards
- Bitwise Logic Operation in Embedded C

Languages

- High Level Languages
 - Easy to develop and update
- C Language
 - Acceptable performance
 - Easy to develop and update
- Low Level Language
 - High performance
 - Not portable



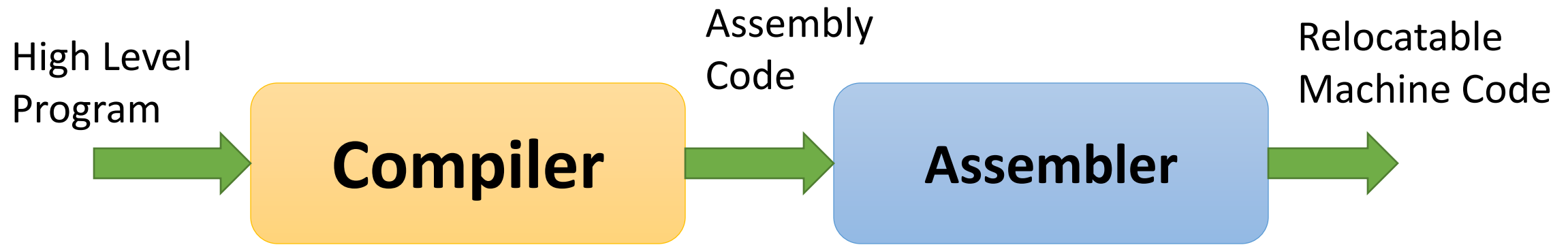
Assembler vs. Compiler

An **assembler** is system software that converts an **assembly language program** (human readable format) into **object code** (machine readable format).

Assembly code (ADD R2, R1, R0) → Machine code (0xEB010200)

A **compiler** is system software that converts a **high-level language program** (human readable format) into **object code** (machine readable format).

C Code (z = x+y) → Assembly code (ADD R2, R1, R0) → Machine code (0xEB010200)



C Code ($z = x + y$) $\rightarrow$ Assembly code (ADD R2, R1, R0) $\rightarrow$ Machine code (0xEB010200)

Structure of a C program

Every C program has a main, and execution begins at the top of this main program. C program has four sections:

1. Documentation
2. Preprocessor directives
3. Global declarations
4. Functions

Example

```
//***** Documentation Section *****  
// This program demonstrated the sections of a C program.  
// Author: Nilufar Poshtiban  
// Date: 4/6/2022  
//*****  
//***** Preprocessor Directives Section *****  
#include <stdio.h>  
#include <stdint.h>  
#include <uart.h>    //user library  
  
/* PORTF data register */  
#define PORTFDAT (*((volatile unsigned int*)0x400253FC))  
  
//***** Global Declaration Section *****  
//This section is not mandatory.  
//***** Function Section *****  
// MAIN: mandatory routine for a C program to be executable  
int main(void){  
    UART_Init0;  
    printf("Hello World");  
    while(1){  
    }  
}
```

Special Characters

Character	Meaning
;	End of statement
:	Defined a label
,	Separate elements of a list
()	Start and end of a parameter list
{ }	Start and end of a compound statement
[]	Start and stop of an array index
" "	Start and stop of a string
"	Start and stop of a character constant

C Data Types for Embedded Systems

C Data Types for Embedded Systems

- In general C programming textbooks, we see ***char***, ***short***, ***int***, ***float***, and ***double*** datatypes.
- We need to examine the size of C data type in the light of 32-bit ARM microprocessor.
- The C standards do not specify the size of data types. The **compiler designers** are free to decide the size of each data type.

C Data types for Embedded Systems

defined by Keil MDK-ARM

Data type	Size	Description
Char	1 byte (D7-D0)	It can be signed (-128 to 127) or unsigned (0 to 255)
Short int (short)	2 bytes (D15- D0)	It can be signed (-32,768 to 32,767) or unsigned (0 to 65,535)
Int	See description • 2 bytes for a 16-bit processor • 4 bytes for a 32-bit processor	Usually represents for the native data size of the processor.
Long int (long)	4 bytes (D31- D0)	It can be signed (-2^{31} to $2^{31}-1$) or unsigned (0x0000,0000 to 0xFFFF,FFFF)
Long long	8 bytes (D63- D0)	It can be signed (-2^{63} to $2^{63}-1$) or unsigned (0x0000,0000,0000,0000 to 0xFFFF,FFFF,FFFF,FFFF)

What is memory alignment?

- A **32-bit processor** (like many ARM cores) is optimized to access memory in chunks of **32 bits = 4 bytes**.
Because of this, it prefers data to be stored at addresses that are multiples of 4:
- Valid aligned addresses end in hex:
...0, ...4, ...8, ...C
- Example:
 - Address 0x1000  aligned
 - Address 0x1002  not aligned to 4 bytes

Unaligned Memory Access

- If a variable is stored so that it **crosses a 4-byte boundary**, accessing it is called **unaligned access**.

Example (`short` = 2 bytes) starts at 0x1003:

Address: 0x1002 0x1003 0x1004 0x1005

This short starts at 0x1003, so part of it is in one 4-byte word and part in the next. That's **unaligned**.

Why is Unaligned Memory Access a problem?

- Because the CPU is built to fetch memory in 4-byte chunks:
- Some ARM processors:
 - **Do not support** unaligned access → causes a **crash (fault)**
- Others:
 - Support it, but:
 - Must perform **two memory reads instead of one**
 - **Slower performance**

Why should I care about which data type to use?

There are three major reasons why a programmer should care about data type:

- Performance
- Overflow
- Coercion

Performance

Using 64-bit data type for saving 32-bit variable will cause:

1. Waste RAM resource
2. Twice RAM access time
3. Additional arithmetic instructions

Overflow

- Unlike assembly language programming, high level language programs do not provide indications when overflow occurs and the program just fails silently.
- If you use a `short int` to hold the number of seconds of a day, the second count will overflow from 32,767 to -32,768. Even if your program handles negative second count, the time will jump back to the day before.
- With 32-bit `int` in a 32-bit ARM processor, overflow is a much less frequent problem because a 32-bit int will hold a number up to 2,147,483,647 but it does not eliminate the problem

Coercion

- If you write a statement with different operand data types for a binary operation, the compiler will convert the smaller data type to the bigger data type. This implicit data type is called **coercion**.
- The compiler may or may not give you warning when coercion occurs.
- If the variable is signed and the data size is increased, the new bits are filled with the sign bit (most significant bit) of the original value.
- When you assign a larger data type to a smaller data type variable, the higher order bits will be truncated.

How to find the exact size of data type?

There are three methods:

1. Read the compiler manual
2. Use pseudo function **sizeof()**
3. Use C99 data types

Data Types in ISO C99 Standard

- Most C programmers use **ANSI C (ISO C89)** data types, but many C programmers are not familiar with **ISO C99** standard.
- In **C standard**, the **size** of integer data types **were not defined** and are **up to the compiler** to decide.
- By convention, `char` is one byte and `short` is two bytes, but `int` and `long` vary greatly among compilers.
- In **ISO C99 standard**, a set of data types are defined with number of bits and sign clearly defined in the data type names.
- These integer data types are defined in a header file called `stdint.h`. You need to include this header file to use these data types.

ANSI C (ISO C89) integer data types and their ranges

Data type	Size	Range Min	Range Max
char	1 byte	-128	127
unsigned char	1 byte	0	255
short int	2 bytes	-32,768	32,767
unsigned short int	2 bytes	0	65,535
int	4 bytes	-2,147,483,648	2,147,483,647
unsigned int	4 bytes	0	4,294,967,295
long	4 bytes	-2,147,483,648	2,147,483,647
unsigned long	4 bytes	0 to 4,294,967,295	
long long	8 bytes	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long long	8 bytes	0	18,446,744,073,709,551,615

ISO C99 integer data types and their ranges

Data type	Size	Range Min	Range Max
int8_t	1 byte	-128	127
uint8_t	1 byte	0 to	255
int16_t	2 bytes	-32,768	32,767
uint16_t	2 bytes	0	65,535
int32_t	4 bytes	-2,147,483,648	2,147,483,647
uint32_t	4 bytes	0	4,294,967,295
int64_t	8 bytes	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
uint64_t	8 bytes	0	18,446,744,073,709,551,615

Note that the range of data types in ISO C99 is the same as ANSI C standard (ISO C89) except it uses more descriptive syntax.

Bit-wise Operations in C

Bit-wise Logic Operations for C

A	B	AND (A & B)	OR (A B)	EX-OR (A ^ B)	Invert ~B
0	0	0	0	0	1
0	1	0	1	1	0
1	0	0	1	1	1
1	1	1	1	0	0

Examples

- `0x35 & 0x0F results in 0x05 /* ANDing*/`
- `0x04 | 0x68 results in 0x6C /* ORing*/`
- `0x54 ^ 0x78 results in 0x2C /* XORing*/`
- `~0x55 results in 0xAA /* Inverting 0x55 */`

Calculations (1/4)

- `0x35 & 0x0F` results in `0x05` /* ANDing */

	00110101	(0x35)
&	00001111	& (0x0F)
	-----	-----
	00000101	(0x05)

Calculations (2/4)

- `0x04 | 0x68` results in `0x6C` /* ORing* /

00000100	(0x04)
01101000	(0x68)
-----	-----
01101100	(0x6C)

Calculations (3/4)

- $0x54 \wedge 0x78$ results in $0x2C$ /* XORing* /

	01010100	(0x54)
$\wedge$	01111000	$\wedge$ (0x78)
	-----	-----
	00101100	(0x2C)

Calculations (4/4)

- `~0x55` results in `0xAA` /* Inverting `0x55` */

~ 01010101

10101010

~ (0x55)

(0xAA)

Bitwise Operation Recap

- **AND (&):** Clears specific bits.
- **OR (|):** Sets specific bits.
- **XOR (^):** Toggles specific bits.
- **NOT (~):** Inverts bits.
- **Shift Left (<<) and Right (>>):** Shifts bit positions.

What is the application?

Why should I care about bitwise Operation?



Applications

1. **Set** a particular bit in a variable or register
2. **Clear** a particular bit in a variable or register
3. **Toggle** a particular bit in a variable or register
4. **Test** a given bit to see if it is high (1) or low (0)

Setting and Clearing (masking) bits

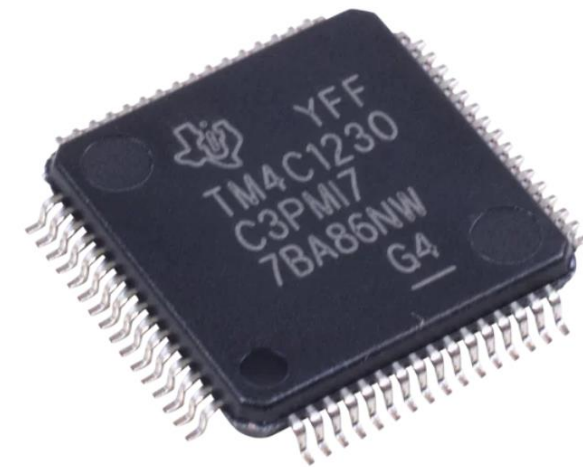
- Anything **ORed** with a 1 results in a 1; anything **ORed** with a 0 results in no change.
- Anything **ANDed** with a 1 results in no change; anything **ANDed** with a 0 results in a zero.
- Anything **EX-ORed** with a 1 results in the complement; anything **EX-ORed** with a 0 results in no change.

TM4C123GH6M Chip

TM4C123GH6M has 64 I/O pins.

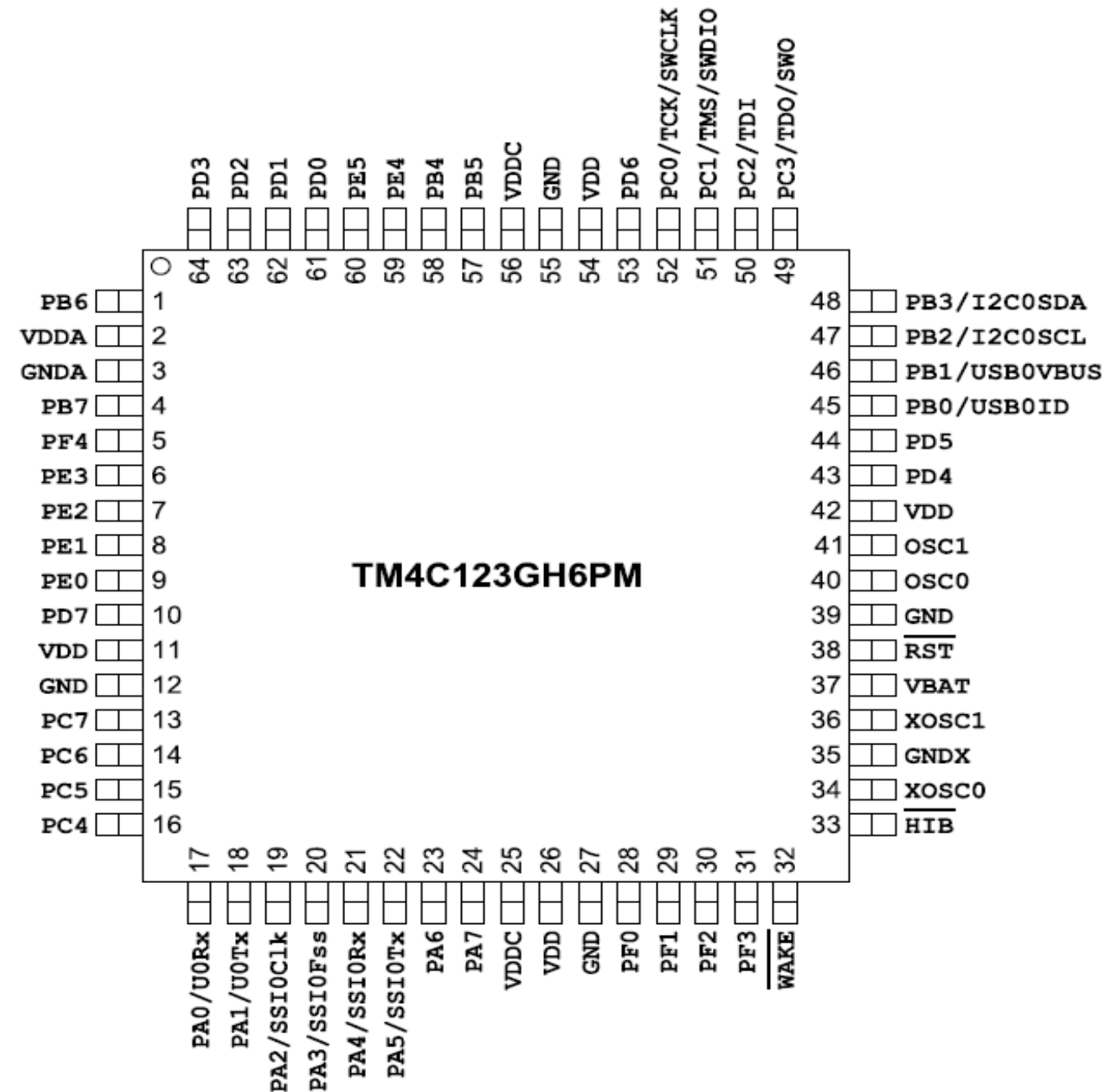
Up to 43 pins are used as GPIO pins that are arranged in 6 ports:

Port Name	Number of Pins
Port A	8 pins
Port B	8 pins
Port C	8 pins
Port D	8 pins
Port E	6 pins
Port F	5 pins



TM4C134GH6PM

Pin-out

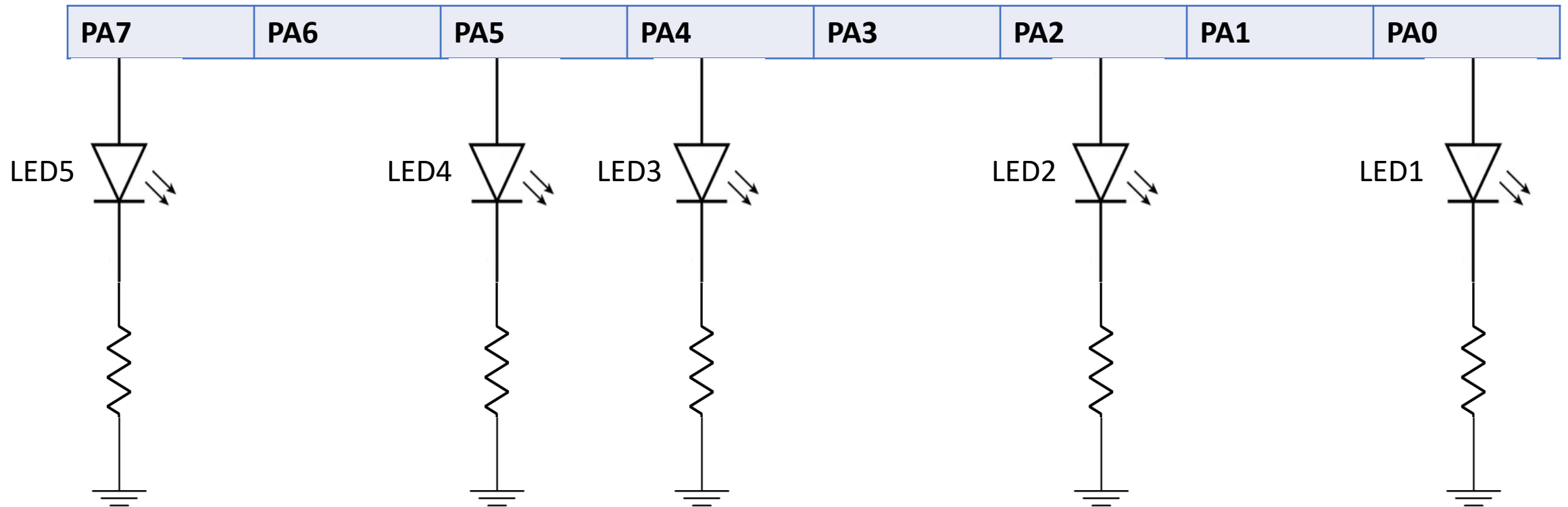


Example – PORT A

PA7	PA6	PA5	PA4	PA3	PA2	PA1	PA0
-----	-----	-----	-----	-----	-----	-----	-----

- Each port is assigned to a register- let's call this one PORTA.
- Each pin is internally connected to a circuit.
- You can program the pin to work as an **input or output**. More about this later!
- For now, we assume that all pins are programmed to work as an **output**.
- **When any pin is set (pin = 1), it provides 5V.**
- **When any pin is cleared (Pin = 0), it provides 0V.**

Example – PORT A



Case 1: Clear all bits of PORTA

```
unsigned int PORTA;  
PORTA = 0x00;
```

- This will write 0 to all bits of PORTA, which will clear all pins.
- The voltage on all pins are now 0V, thus all LEDs are OFF.

Case 2: Turn on all LEDs

```
unsigned int PORTA;
```

```
PORTA = 0x00;
```

```
PORTA = 0b1011,0101; or PORTA = 0xB5;
```

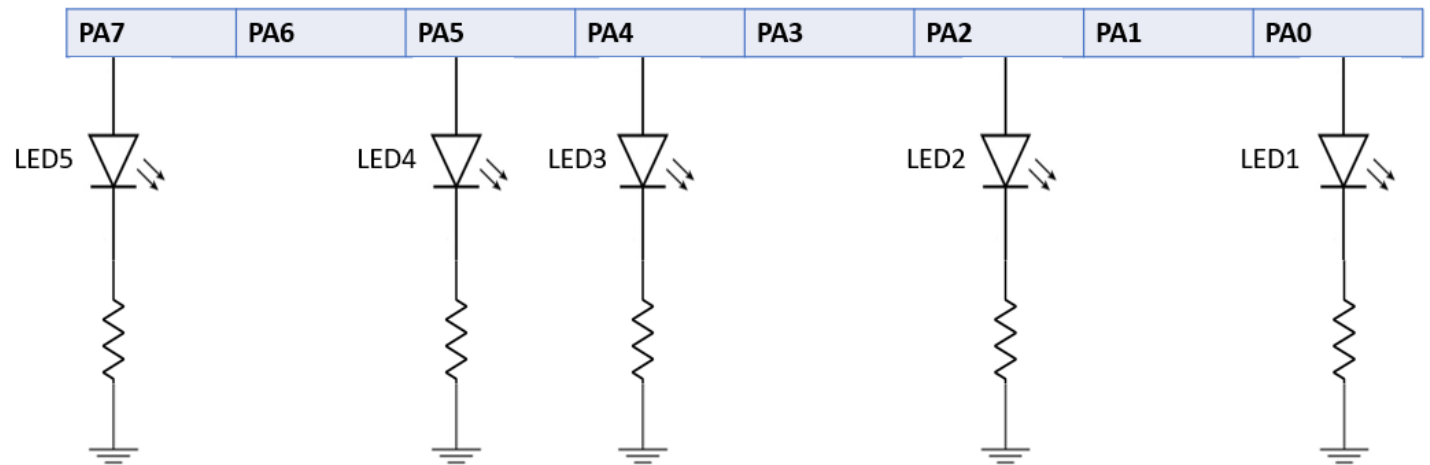
PA0 = 1 (5V) → LED1: ON

PA2 = 1 (5V) → LED2: ON

PA4 = 1 (5V) → LED3: ON

PA5 = 1 (5V) → LED4: ON

PA7 = 1 (5V) → LED5: ON



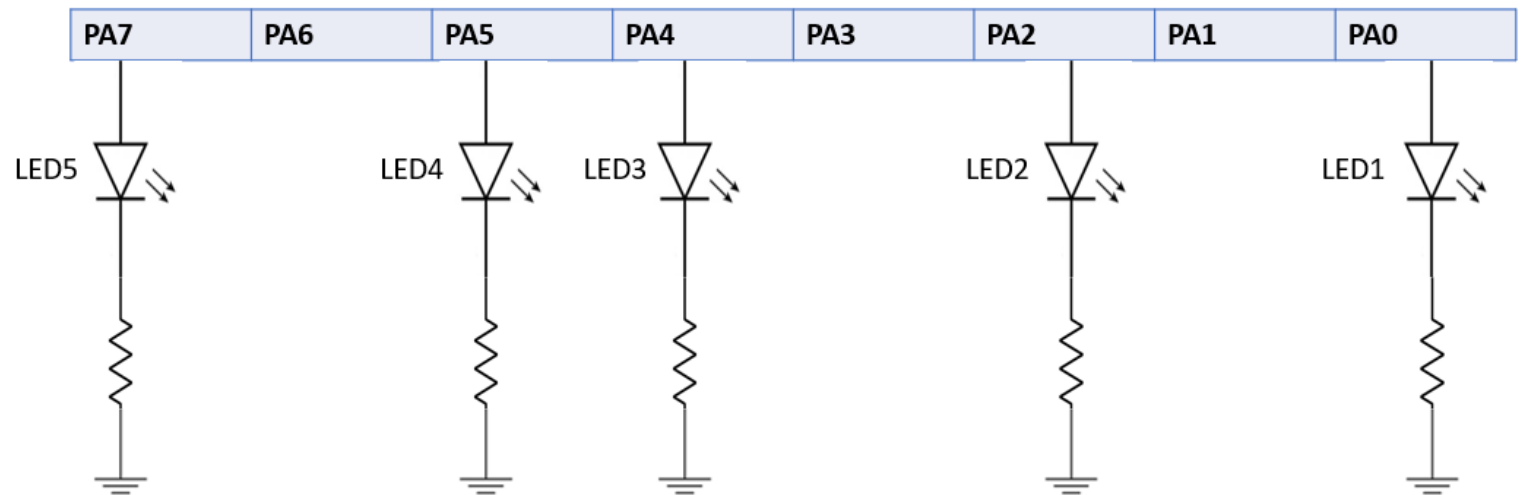
Let's look at some cases where we need to **clear a bit**

We will use **bitwise AND** to clear a desired bit!

Case 3: Turn off LED1 without changing others

```
unsigned int PORTA;  
PORTA = 0x00;  
PORTA = 0b1011,0101; or PORTA = 0xB5;
```

We need to **clear**
PA0 bit:
(i.e, PA0 = 0 (0V))
but keep all other
bits unchanged.



Case 3: Turn off LED1 without changing others

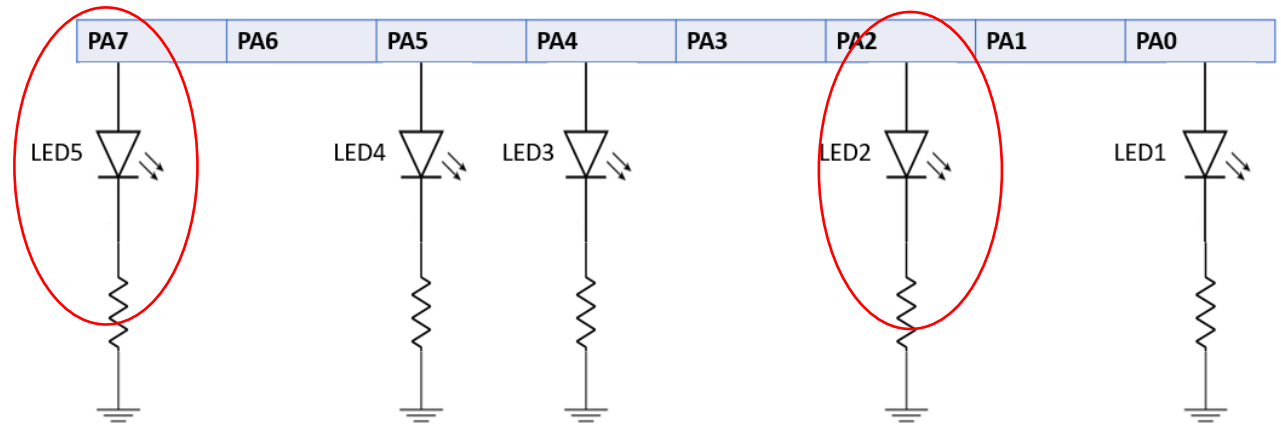
```
unsigned int PORTA;  
PORTA = 0x00;  
PORTA = 0b1011,0101; or PORTA = 0xB5;  
PORTA = PORTA & 0b1111,1110; or PORTA &= 0xFE
```

This will **clear** bit0 of PORTA :
PA0 = 0 (0V), thus LED1 turns off.

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Case 4: Add to the code to turn off LED 2 and LED 5, keep all others unchanged

```
unsigned int PORTA;  
PORTA = 0x00;  
PORTA = 0b1011,0101; or PORTA = 0xB5;  
PORTA = PORTA & 0b1111,1110;
```

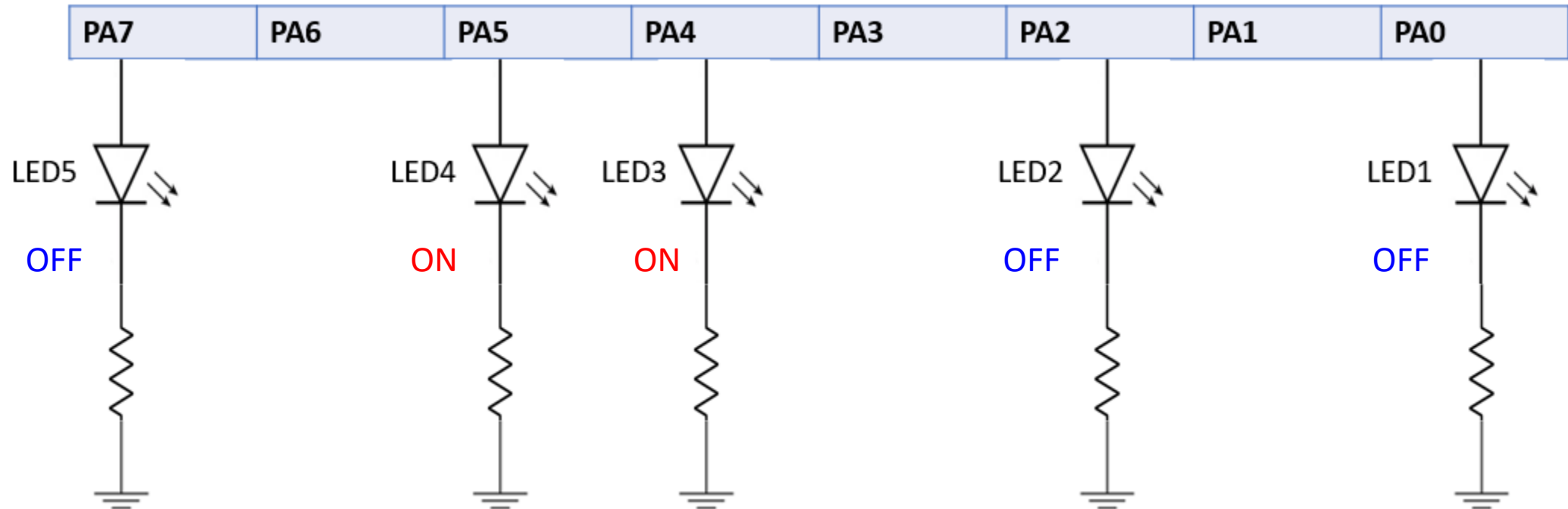


Case 4: Add to the code to turn off LED 2 and LED 5, keep all others unchanged

```
unsigned int PORTA;  
PORTA = 0x00;  
PORTA = 0b1011,0101; PORTA = 0xB5;  
PORTA = PORTA & 0b1111,1110; or PORTA &= 0xFE;  
PORTA = PORTA & 0b0111,1011; or PORTA &= 0x7C;
```

- Now LED 2, LED5 turn OFF.
- LED1 (OFF), LED3 (ON) and LED4 (ON) remain unchanged.

Current State of LEDs



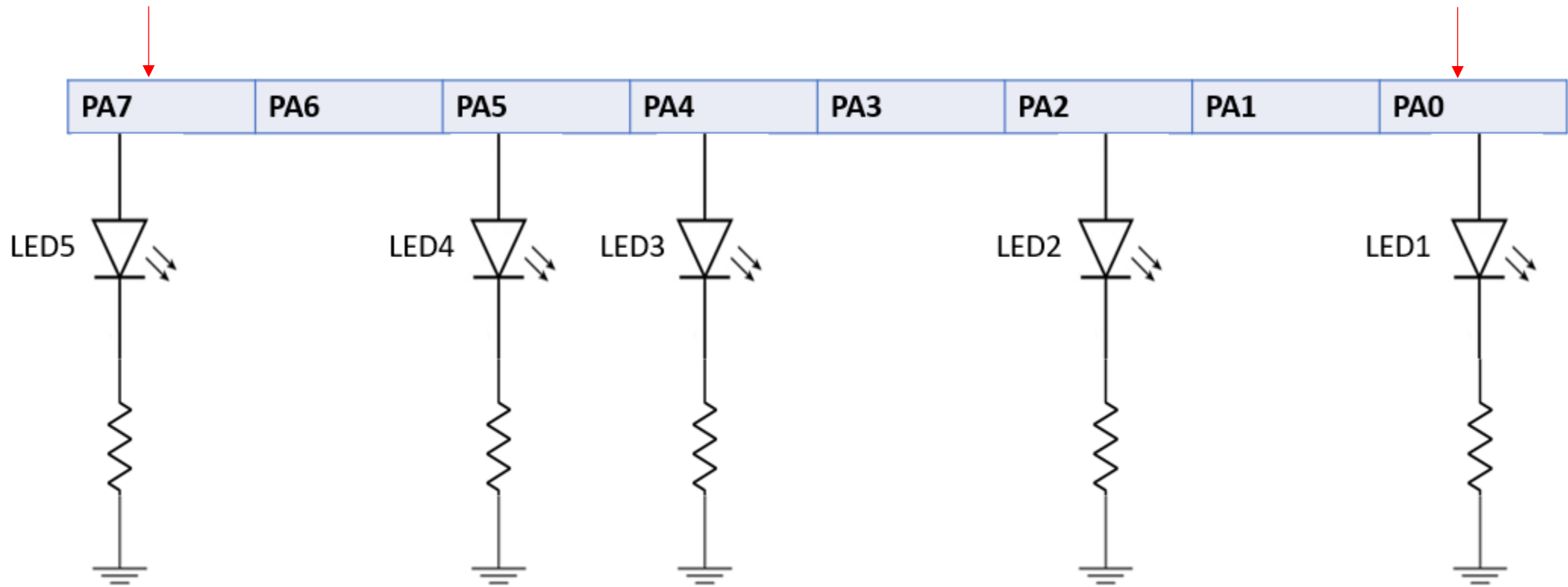
Let's look at a case where we need to **set a bit**

We will use **bitwise OR** to set a desired bit!

Case 5: Turn on LED 1 and LED 5 without changing other pins

Change this bit to 1

Change this bit to 1



Case 5: Turn on LED 1 and LED 5 without changing other pins

```
unsigned int PORTA;
```

```
PORTA = PORTA | 0b1000,0001; or PORTA |= 0x81
```

This will set bit0 (PA0) and bit7 (PA7), thus LED 1 and LED 5 will turn on.

The other LEDs remain unchanged

(i.e. LED2: OFF, LED3: ON, LED 4: ON (refer to slide 42).

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

This time, we will look at a case where we need to **toggle a bit**

We will use **bitwise XOR** to toggle a desired bit!

Case 6: The following program **toggles only LED3** of PORTA continuously without disturbing the rest of the LEDs (bits).

```
void main(void) {  
    while(1) {  
        unsigned int PORTA;  
        PORTA ^= 0b0001,0000; or PORTA ^= 0x10  
    }  
}
```

A	B	A ^ B
0	0	0
0	1	1
1	0	1
1	1	0

Let's look at the cases where we need to **test a bit**

We use bitwise AND and a MASK number to test a bit

Introduction to Bit Testing

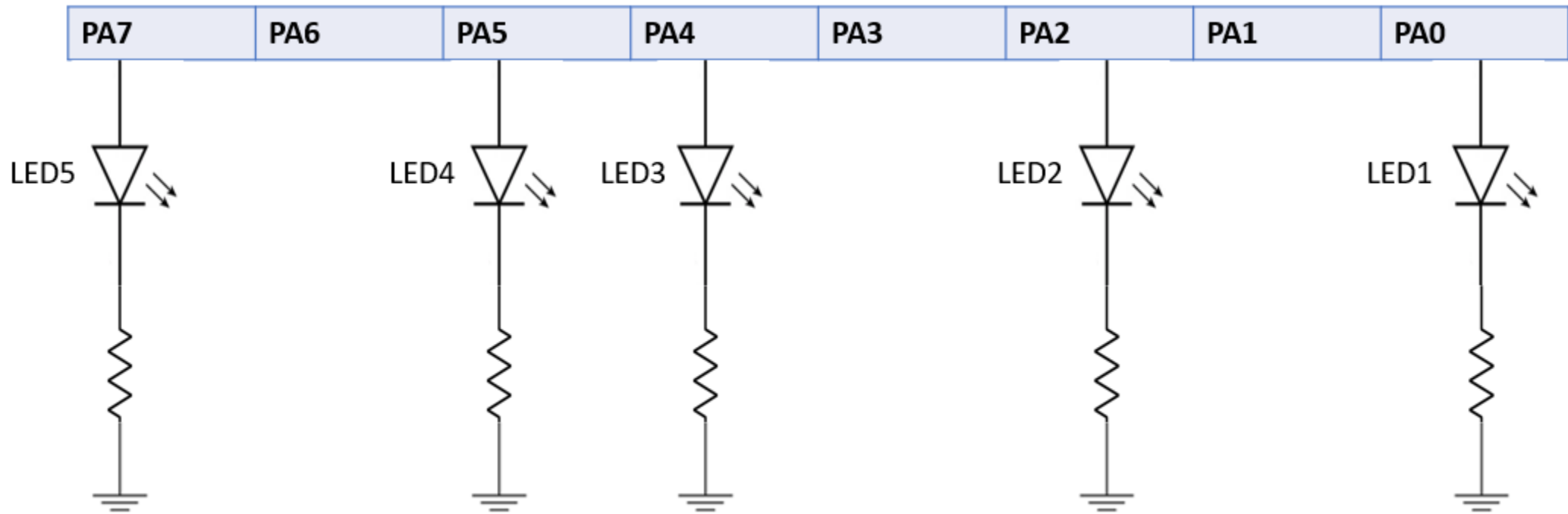
What is Bit Testing?

- Checking whether a specific bit in a binary number is **set (1)** or **clear (0)**.

Why Use Bit Testing?

- Efficiently check the status of hardware registers or flags.
- Control and monitor individual features in microcontrollers.

Example 1: If PA1 of PORTA is HIGH, turn on LED 1



Example 1: If PA1 (bit 1) of PORTA is HIGH, turn on LED 1

```
void main() {  
    unsigned int PORTA;  
    if (PORTA & 0b0000,0010) or if (PORTA & 0x02)  
    {  
        PORTA = PORTA | 0b0000,0001; set bit0 of PORTA  
    }  
}
```

Example 1: If PA1 (bit 1)

```
void main() {  
    unsigned int PORTA;  
    if (PORTA & 0b0000,0010) or if (PORTA & 0x02)  
    {  
        PORTA = PORTA  
    }  
}
```

If PA1 = 1, then the value of PORTA = 0bxxxx,xx1x

0bxxxx,xx1x
& 0b0000,0010

0b0000,0010 (if condition: (True))

If PA1 = 0, then the value of PORTA = 0bxxxx,xx0x

0bxxxx,xx0x
& 0b0000,0010

0b0000,0000 (if condition:
(False))

Example

Write a C program to monitor bit 4 of var1. If it is HIGH, change the value of var2 to 0x55; otherwise, change the value of var 2 to 0xAA.

...

```
While(1)
```

```
{
```

```
    if (var1 & 0x10)    /* check bit 4 of var1 */
```

```
        var2 = 0x55; /* This statement is executed if bit 4 is 1 */
```

```
    else
```

```
        var2 = 0xAA; ; /* This statement is executed if bit 4 is 0 */
```

```
}
```

...

Compound Operators

Instruction	Its equivalent using compound operators
<code>a = a + 6;</code>	<code>a += 6;</code>
<code>a = a - 23;</code>	<code>a -= 23;</code>
<code>y = y * z;</code>	<code>y *= z;</code>
<code>z = z / 25;</code>	<code>z /= 25;</code>
<code>w = w 0x20;</code>	<code>w = 0x20;</code>
<code>v = v & mask;</code>	<code>v &= mask;</code>
<code>m = m ^ togBits;</code>	<code>m ^= togBits;</code>

Bit-wise shift operation in C

Operation	Symbol	Format of Shift Operation
Shift Right	>>	data >> number of bit-positions to be shifted right
Shift Left	<<	data << number of bit-positions to be shifted left

Example:

```
0b00010000 >> 3    /* it equals 0b0000010. Shifting right 3 times */
```

```
0b00010000 <<3     /* it equals 0b1000000. Shifting left 3 times */
```

```
1<<3               /* it equals 0b0001000. Shifting left 3 times */
```

Using shift operator to generate mask

One way to ease the generation of the mask is to use the left shift operator. To generate a mask with **bit n set to 1**, use the expression: **$1 \ll n$** .

Example:

To set bit 3 of the register, we can write:

```
register |= 0x08;    or    register |= 1<<3;
```

Using shift operator to generate mask (*contd.*)

If more bits are to be set in the mask, they can be “or” together. To generate a mask with bit n and bit m set to 1, use the expression:

$(1 \ll n) \mid (1 \ll m)$

Example:

To set bit 6 and bit 1 of the register, we can write:

```
register |= 0x42; or Register |= (1<<6) | (1<<1);
```

Using shift operator to generate mask (*contd.*)

To clear bit(s) of a register, we can use

```
register &= ~(1<<n);
```

Example:

To clear bit 5 of the register, we can write:

```
register &= 0xDF; or register &= ~(1<<5);
```

Example

```
register &= ~(1 << 4) | (1 << 1);
```

Clears bits 4 and 1 of the register.

Recap- Bitwise Operations

Bit-wise operations using compound operators

The majority of hardware access level code involves:

- setting a bit or bits in a register
- clearing a bit or bits in a register
- toggling a bit or bits in a register and
- monitoring the status bits.

*For the first three cases, the operation read the content of the register, modify a bits then write it back to the same register. The compound operators are very suitable.

To set bit(s) in a register

`register |= MASK;`

Where MASK is a number that has '1' at the bit(s) to be set.

Examples

Register |= 0x08

- The number 0x08 has a '1' at bit 3, therefore the statement sets bit 3 of the register.

Register |= 0x42

- The number 0x42 has a '1' at bit 6 and bit 1, therefore the statement sets bit 6 and 2 of the register.

To clear bit(s) in a register

`register &= ~MASK;`

Where MASK is a number that has '1' at the bits to be cleared.

Examples

Register $\&= \sim 0x20$

- The number 0x20 has a '1' at bit 5, therefore the statement clears bit 5 of the register.

Register $\&= \sim 0x12$

- The number 0x12 has a '1' at bit 4 and bit 1, therefore the statement clears bit 4 and 2 of the register.

To toggle bit(s) in a register

`register ^= MASK;`

Where the bits with '1' in MASK will be toggled, the bits with '0' in MASK will not change,

Examples

Register ^= 0x04

- The number 0x04 has a '1' at bit 2, therefore the statement toggles bit 2 of the register.

Register |= 0x82

- The number 0x82 has a '1' at bit 7 and bit 1, therefore the statement toggles bit 7 and 2 of the register.

Reference

Mazidi, M., Naimi, S., Naimi, S. (2021). The STM32F103 ARM Microcontroller and Embedded Systems, Using Assembly and C. MicroDigitalEd. ISBN13 9781970054019